Programação Orientada a Objetos

CONSTRUTOR DE CÓPIA

em C++

Introdução

- Projetamos uma Classe String
  - Com alocação dinâmica de memória
    - Construtor: aloca memória
    - Destrutor: libera memória

```
int main()
{
    String nome { "C++" };
}
```

```
class String
{
private:
   char * str;
   size_t tam;
   ...
};
```

```
// construtor
String::String(const char txt[])
{
    tam = strlen(txt);
    str = new char[tam + 1];
    strcpy(str, txt);
}
```

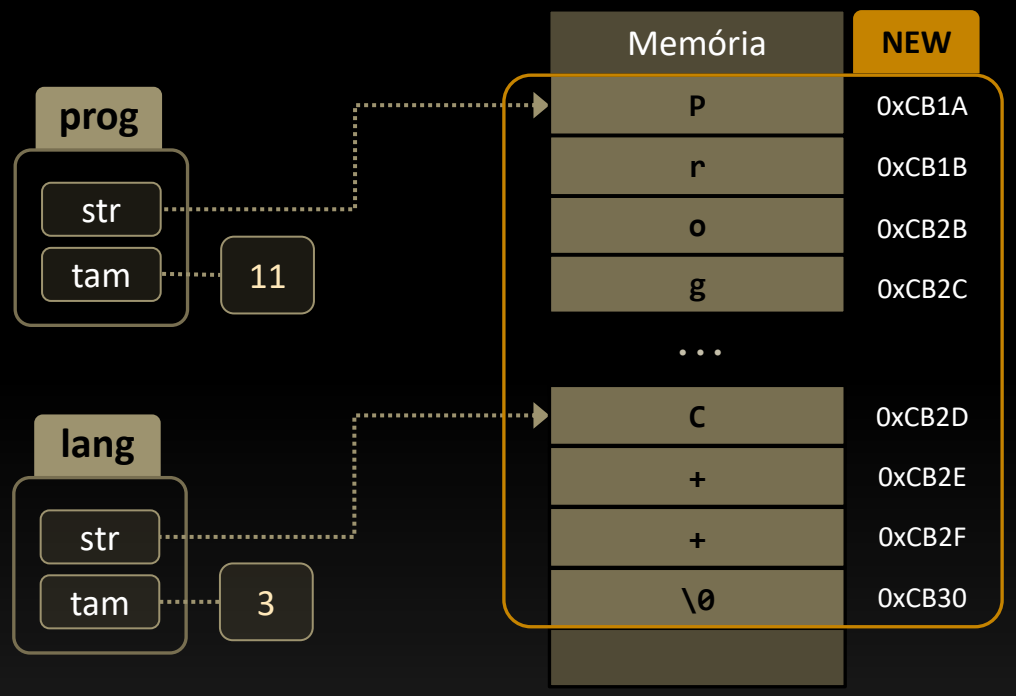
```
// destrutor
String::~String()
{
    delete [] str;
}
```

Introdução

- A Classe String permite:
  - Criar novos objetos
  - Exibir texto

```
int main()
{
    String prog { "Programação" };
    String lang { "C++" };

    cout << prog << endl;
    cout << lang << endl;
}
```



Introdução

- Vamos expandir o uso da classe String
  - Passando objetos para funções

```
int main()
{
    String lang { "C++" };

    Valor( lang );

    cout << lang << endl;
}

void Valor(String s)
{
    cout << "Valor: " << s << "\n";
}
```

Passagem por valor: copia conteúdo para o parâmetro da função

Introdução

- Saída do programa:

```
-------------| Objeto (1) criado: "C++"
Valor: C++
-------------| Objeto (1) deletado: "C++", 0 objeto(s) ainda vivo(s)
♦♦♦♦♦♦♦♦♦B#m
-------------| Objeto (0) deletado: "♦♦♦♦♦♦♦♦♦B#m",
```

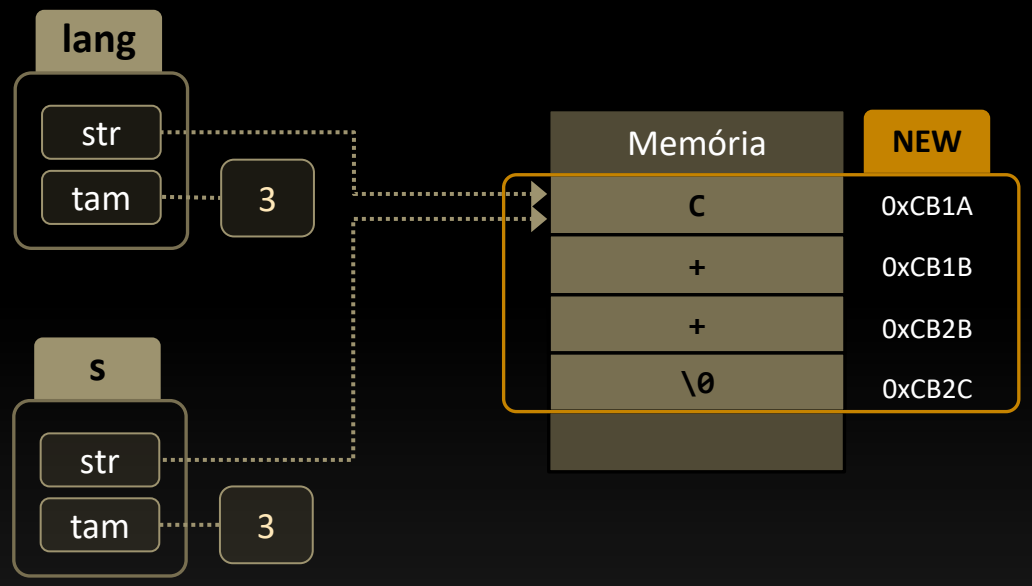
  - O programa foi fechado abruptamente
    - Um objeto criado
    - Dois objetos destruídos
    - Exibição de lixo

Cópia de Objetos

- Existem problemas na cópia de objetos
  - Os membros são copiados
  - Não a memória apontada

```
int main()
{
    String lang { "C++" };
    Valor( lang );
}

void Valor(String s)
{
    cout << "Valor: " << s << "\n";
}
```



Cópia de Objetos

- A passagem de parâmetro faz uma cópia
  - É equivalente a uma inicialização

```
int main()
{
    String lang { "C++" };

    Valor( lang );
}
```

Inicializa s com lang

```
void Valor(String s)
{
    cout << "Por Val: " << s << "\n";
}
```

```
int main()
{
    String lang { "C++" };

    String s { lang };

    // equivalentes
    String s (lang);
    String s = lang;
    String s = String(lang);
}
```

Inicializa s com lang

Todas invocam o construtor de cópia

Cópia de Objetos

- O construtor de cópia:
  - Inicializa um objeto a partir de outro
  - É criado automaticamente pelo compilador

```
int main()
{
    String lang { "C++" };

    String s { lang };
}
```

```
String s = String(lang);
```

```
// construtor de cópia criado
// automaticamente pelo compilador

String::String(const String& param)
{
    tam = param.tam;
    str = param.str;
}
```

Apenas copia atributos,
não aloca memória

Solução do Problema

- Precisamos definir um construtor de cópia
  - Quando aquele criado pelo compilador não atender
  - Ao utilizar alocação dinâmica de memória

```
int main()
{
    String lang { "C++" };

    String s { lang };
}
```

```
String s = String(lang);
```

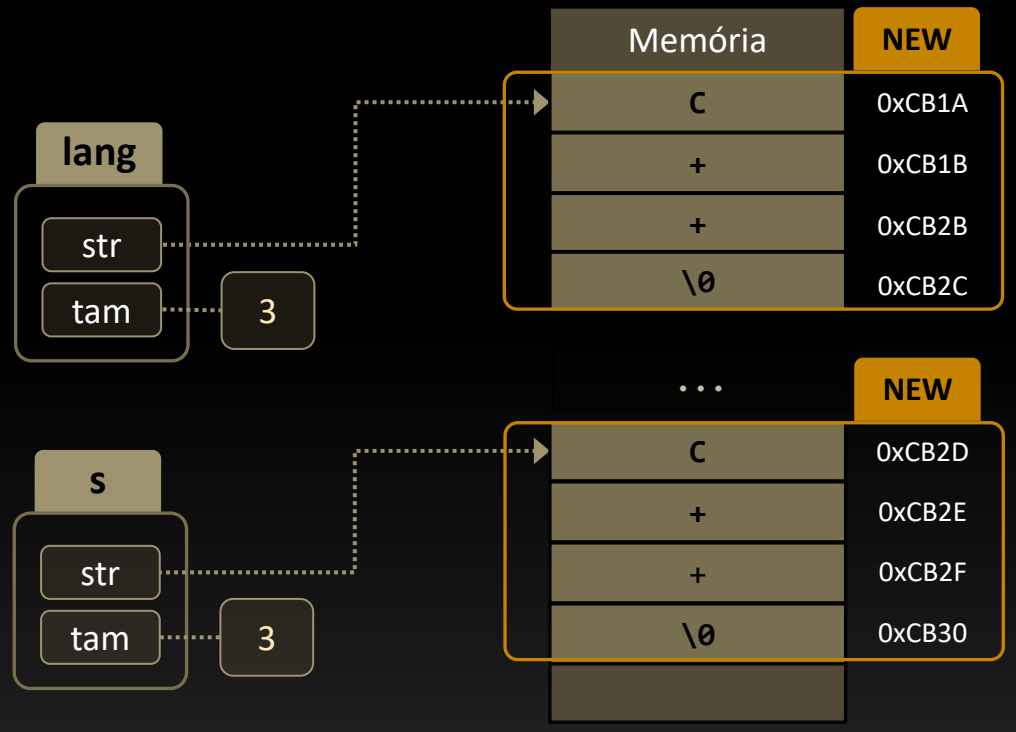
```
// construtor de cópia
String::String(const String& param)
{
    tam = param.tam;
    str = new char[tam + 1];
    strcpy(str, param.str);
    objs++;
}
```

Solução do Problema

- Um construtor de cópia deve sempre ser definido?
  - É uma boa prática de programação

```
int main()
{
    String lang { "C++" };
    String s { lang };
    ...
```

```
String::String(const String& param)
{
    tam = param.tam;
    str = new char[tam + 1];
    strpcy(str, param.str);
    objs++;
}
```



Métodos Especiais

- C++ define algumas funções membro especiais
  - Construtor padrão
  - Construtor de cópia
  - Construtor de transferência
  - Destrutor padrão
  - Operador de atribuição
  - Operador de atribuição com transferência
  - Operador de endereço
- Criados automaticamente pelo compilador quando:
  - São necessárias no programa
  - E não foram definidas pelo programador

Construtor Padrão

- **C++** fornece um construtor padrão
  - Caso um não seja fornecido

```
// definido pelo compilador
String::String()
{ }
```

  - Ele é necessário para criar objetos

```
String nome;  // chama o construtor padrão
```

  - O construtor fornecido cria um objeto não inicializado
    - Similar a variáveis automáticas

```
int num;      // variável não inicializada
String nome;  // variável não inicializada
```

Construtor Padrão

- C++ **não** fornece um construtor padrão
  - Caso o programador defina um
    - O construtor padrão não recebe argumentos

```
// definido pelo programador
String::String()
{ }
```

    - Pode ser criado com argumentos padrão
      - Se todos os argumentos possuírem valores

```
String::String(int tam = 0)
{

}
```

Construtor de Cópia

- Um construtor de cópia inicializa um objeto
  - A partir de outro existente (da mesma classe)

Nome da Classe

Referência

```
Classe( const Classe & );
```

Constante

  - Exemplos:

```
String(const String&);    // construtor de cópia
Canvas(const Canvas&);    // construtor de cópia
Jogador(const Jogador&);  // construtor de cópia
Notepad(const Notepad&);  // construtor de cópia
```

Construtor de Cópia

- Ele é utilizado nas seguintes situações:
  - Inicialização explícita

```
String s { lang };          // C++11
String s (lang);            // C++98
String s = lang;            // tradicional
String s = String(lang);    // explícita
```

  - Passagem e retorno de valor

```
String Transforma(String s);
```

  - Criação de temporários

```
s = prog + comp + lang;
```

String

temporário sem nome

Considerando os objetos

```
String prog { "Programação" };
String comp { "Computador" };
String lang { "C++" };
```

E a existência de um operador + para concatenar Strings.

Construtor de Cópia

- Ele **não** é utilizado em:
  - Atribuições

```
String lang { "C++" };
String s = lang;    // utiliza construtor de cópia
s = lang;           // utiliza operador de atribuição
```

  - Passagem por referência

```
int main()
{
    String lang { "C++" };
    Refer( lang );
}
```

```
void Refer(String& s)
{
    cout << "Referência: " << s << "\n";
}
```

A Classe String

- Atualizando o exemplo:

```
class String
{
private:
   char * str;                    // ponteiro para string
   size_t tam;                    // tamanho da string
   static int objs;               // número de instâncias

public:
   String(const char txt[]);      // construtor
   String(const String& s);       // construtor de cópia
   String();                      // construtor padrão
   ~String();                     // destrutor

   // função amiga para exibir String com cout
   friend ostream& operator<<(ostream& os, const String& s);
};
```

» String.h

A Classe String

```cpp
#include "String.h"
#include <cstring>

int String::objs = 0;

String::String(const char txt[])
{
    tam = strlen(txt);
    str = new char[tam + 1];
    strpcy(str, txt);
    objs++;
}

String::String()
{
    tam = 0;
    str = new char[1] { "" };
    objs++;
}
```

```cpp
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

public:
   String(const char txt[]);
   String(const String& s);
   String();
   ~String();

   friend ostream& operator<<(
      ostream& os, const String& s);
};
```

» String.cpp

A Classe String

```
String::String(const String& s)
{
    tam = s.tam;
    str = new char[tam + 1];
    strpcy(str, s.str);
    objs++;
}

String::~String()
{
    delete [] str;
    objs--;
}

ostream& operator<<(ostream& os, const String& s)
{
    os << s.str;
    return os;
}
```

```
class String
{
private:
   char * str;
   size_t tam;
   static int objs;

public:
   String(const char txt[]);
   String(const String& s);
   String();
   ~String();

   friend ostream& operator<<(
      ostream& os, const String& s);
};
```

» String.cpp

A Classe String

```
void Refer(String& s) { cout << "Refer: " << s << "\n"; }

void Valor(String  s) { cout << "Valor: " << s << "\n"; }

int main()
{
     String atual { "Torre Eiffel completa 137 anos" };
     String tecno { "Gmail agora permite mudar endereço" };

     Refer(atual);
     cout << "Atual: " << atual << endl;

     Valor(tecno);
     cout << "Tecno: " << tecno << endl;

     String breaking = tecno;
     cout << "Break: " << breaking << endl;
}
```

» Noticias.cpp

A Classe String

- Saída do programa:

```
Refer: Torre Eiffel completa 137 anos
Atual: Torre Eiffel completa 137 anos

Valor: Gmail agora permite mudar endereço
Tecno: Gmail agora permite mudar endereço

Break: Gmail agora permite mudar endereço
```

  - O construtor de cópia foi chamado 2 vezes

```
Valor(tecno);
String breaking = tecno;
```

Resumo

- Ao utilizar alocação dinâmica de memória
  - Devemos fornecer:
    - Construtor: alocar memória
    - Destrutor: liberar memória
    - Construtor de cópia: copiar memória alocada
- O construtor de cópia é usado:
  - Inicialização com objeto existentes
  - Passagem e retorno de objetos (por valor)
  - Criação de objetos temporários